PostgreSQL

你在系統設計面試裡很有可能會討論到 PostgreSQL。畢竟它在 Stack Overflow 開發者調查中長期位居最受喜愛資料庫的榜首，Reddit、Instagram，甚至你現在正在閱讀的這個網站都在使用它。

話雖如此，有一件事很重要：你的面試官不是在找一個資料庫管理員。他們想看到你能做出有依據的架構決策。什麼時候該選 PostgreSQL？什麼時候該看看別的選項？有哪些關鍵的取捨要考慮？

我常看到面試者在這裡卡關。他們可能鑽得太深，開始聊 MVCC 和 WAL 的內部實作——但面試官只是想知道它能不能處理他們的資料關係；要嘛就做出過於籠統的論斷，比如「NoSQL 的擴展性比 PostgreSQL 好」，卻沒有真正理解其中的細微差異。

這篇講義會聚焦在系統設計面試中你真正需要知道的 PostgreSQL 知識。我們會從一個實際的例子出發，探索應該影響你決策的關鍵能力與限制，然後帶到常見的面試場景。

如果你對 SQL 還不熟悉，本文末尾有一個附錄：基礎 SQL 概念，可以先去看一下。

一個帶出直覺的例子

讓我們用一個具體的例子來建立對 PostgreSQL 的直覺。假設我們在設計一個社群媒體平台。不是像 Facebook 那種規模的龐然大物，而是一個正在成長、需要穩固基礎的平台。

這個平台需要處理幾個基本的關係：

- 用戶可以建立貼文
- 用戶可以對貼文留言
- 用戶可以追蹤其他用戶
- 用戶可以對貼文和留言按讚
- 用戶可以和其他用戶建立私訊（DM）

這正是面試中會出現的那種場景。實體之間的關係清楚但不簡單，而且有一些關於資料一致性和擴展性的有趣問題。

從資料庫的角度來看，為什麼這個例子很有意思？因為不同的操作有不同的需求：

- **多步驟操作**（例如建立私訊對話）需要是原子性的：建立對話串、加入參與者、儲存第一則訊息，這三件事必須一起發生
- **留言和追蹤關係**需要參照完整性：不能有一則留言指向一篇不存在的貼文，也不能去追蹤一個不存在的用戶
- **按讚數**可以是最終一致的：晚個幾秒更新是可被接受的
- **用戶個人頁面**的請求，需要高效率地抓取最近的貼文、追蹤者數量和其他 metadata
- 用戶需要能夠搜尋貼文和找到其他用戶
- 隨著平台成長，需要處理更多資料和更複雜的查詢

這些需求的組合——複雜的關係、混合的一致性需求、搜尋能力、以及成長空間——讓它成為探索 PostgreSQL 優勢與限制的完美例子。在整篇講義裡，我們都會回到這個例子，讓討論落地在實際場景上。

buildmoat.org

核心能力與限制

有了這個帶出直覺的例子，讓我們深入 PostgreSQL 能做好什麼、以及不擅長什麼。大多數關於 PostgreSQL 的系統設計討論，都會圍繞在它的讀取效能、寫入能力、一致性保證和複製機制上。理解這些核心特性，將幫助你做出有依據的決策。

讀取效能

先從讀取效能說起——這很關鍵，因為在大多數應用程式裡，讀取的次數遠遠多於寫入。在我們的社群媒體例子裡，用戶瀏覽貼文和個人頁面的時間，遠多於他們建立新內容的時間。

在系統設計面試中，你不需要深入查詢規劃器（query planner）的內部原理。把注意力放在實際的效能模式上，以及什麼情況下不同類型的索引才說得通。

基礎索引

提升 PostgreSQL 讀取效能最基本的方式就是索引。PostgreSQL 預設使用 **B-tree 索引**，非常適合：

- 精確匹配（`WHERE email = 'user@example.com'`）
- 範圍查詢（`WHERE created_at > '2024-01-01'`）
- 排序（`ORDER BY username`，前提是排序的欄位符合索引欄位的順序）

PostgreSQL 會自動在主鍵欄位上建立 B-tree 索引，但你也可以在其他欄位上自行建立：

```
-- 最基本、最常用的索引
CREATE INDEX idx_users_email ON users(email);

-- 針對常見查詢模式的複合索引
CREATE INDEX idx_posts_user_date ON posts(user_id, created_at);
```

面試中常見的一個陷阱，是建議對每一個欄位都加上索引。記住，每個索引都會：

- **讓寫入變慢**（因為索引也需要跟著更新）
- **佔用磁碟空間**
- **不一定會被用到**（如果查詢規劃器判斷做全表掃描更快，就不會走索引）

進階索引類型

PostgreSQL 真正出色的地方，在於它支援多種特殊用途的索引。這些索引在系統設計面試中常常出現，因為它們往往可以省掉另一個獨立的專用資料庫：

GIN 索引做全文搜尋。PostgreSQL 內建了全文搜尋能力，透過 GIN（Generalized Inverted Index）索引實現。GIN 索引的運作方式像一本書末尾的索引——它記錄每個詞彙出現在哪些位置，讓你能快速找到包含特定關鍵字的文件：

```
-- 為貼文加上 tsvector 欄位
ALTER TABLE posts ADD COLUMN search_vector tsvector;
CREATE INDEX idx_posts_search ON posts USING GIN(search_vector);

-- 現在可以做全文搜尋了
```

buildmoat.org

```
SELECT * FROM posts
WHERE search_vector @@ to_tsquery('postgresql & database');
```

對很多應用程式來說，這個內建的搜尋能力就代表你不需要額外跑一個 Elasticsearch 叢集。它支援詞幹處理（finding/find/finds 都能匹配）、相關性排名、多語言、以及 AND/OR/NOT 的複雜查詢。

不過，PostgreSQL 的全文搜尋並不能在所有情況下取代 Elasticsearch。如果你需要更精細的相關性評分、分面搜尋、模糊匹配、在超大型資料集上做分散式搜尋、或進階的聚合分析，Elasticsearch 才是合適的選擇。對於簡單的搜尋需求，先用 PostgreSQL 內建的能力；只有當 PostgreSQL 確實無法滿足時，才引入 Elasticsearch。架構越簡單越好。

JSONB 欄位搭配 GIN 索引，在你需要對貼文附加靈活 metadata 時特別有用。

JSONB 和 JSON 都儲存 JSON 格式的資料，但 JSONB 以二進位格式儲存並支援索引，實務上幾乎都選 JSONB。

比如在我們的社群媒體平台，每篇貼文可能有不同的屬性——地點、@到的用戶、hashtag、附帶的媒體。與其為每種可能性新增欄位，不如把這些存在 JSONB 欄位裡（給了我們和 NoSQL 資料庫一樣的靈活性！）：

```
-- 為貼文加上 JSONB 的 metadata 欄位
ALTER TABLE posts ADD COLUMN metadata JSONB;
CREATE INDEX idx_posts_metadata ON posts USING GIN(metadata);

-- 現在可以高效率地查詢有特定 metadata 的貼文
SELECT * FROM posts
WHERE metadata @> '{"type": "video"}'
  AND metadata @> '{"hashtags": ["coding"]}';

-- 或者找出所有 @提到特定用戶的貼文
SELECT * FROM posts
WHERE metadata @> '{"mentions": ["user123"]}';
```

buildnoat.org

PostGIS 做地理空間查詢。雖然不是 PostgreSQL 核心的一部分，但 PostGIS 擴充套件加入了強大的空間計算能力。就像我們可以索引文字做快速搜尋，PostGIS 讓我們索引位置資料做高效率的地理查詢。在我們的社群媒體平台上，這讓我們能顯示用戶附近的貼文：

```
-- 啟用 PostGIS
CREATE EXTENSION postgis;

-- 為貼文加上地點欄位（使用 geography 類型，距離單位為公尺）
ALTER TABLE posts
ADD COLUMN location geography(Point, 4326);

-- 建立空間索引
CREATE INDEX idx_posts_location
ON posts USING GIST(location);

-- 找出某用戶 5 公里範圍內的所有貼文
```

```
SELECT * FROM posts
WHERE ST_DWithin(
    location,
    ST_MakePoint(-122.4194, 37.7749)::geography,  -- 舊金山座標
    5000  -- 5 公里，單位公尺
);
```

PostGIS 的能力相當驚人——它能處理不同類型的空間資料（點、線、多邊形）、各種距離計算、空間運算（交集、包含），以及不同的座標系統。Uber 早期甚至用 PostGIS 跑了整個配車系統，後來才因為規模問題換成自訂解法。這足以說明在你需要專門的地理空間資料庫之前，PostgreSQL 能帶你走多遠。

最精彩的是，你可以把這些能力全部組合在一起，創造豐富的搜尋體驗。例如，找出舊金山 5 公里範圍內、內容提到「food」、且標記為「restaurant」hashtag 的所有影片貼文：

```
SELECT * FROM posts
WHERE search_vector @@ to_tsquery('food')
  AND metadata @> '{"type": "video", "hashtags": ["restaurant"]}'
  AND ST_DWithin(
    location,
    ST_MakePoint(-122.4194, 37.7749)::geography,
    5000
  );
```

查詢優化進階技巧

除了選對索引類型，還有一些進階的索引策略能大幅改善讀取效能。

涵蓋索引（Covering Index）

PostgreSQL 用索引找到一筆資料後，通常需要做兩件事：在索引裡查到那筆資料的位置，然後再去表格裡把你需要的其他欄位抓回來。但如果我們能把需要的所有資料都直接存在索引裡呢？這就是涵蓋索引：

```
-- 假設這是社群媒體 App 裡很常見的查詢：
SELECT title, created_at
FROM posts
WHERE user_id = 123
ORDER BY created_at DESC;

-- 涵蓋索引，把所有需要的欄位都包含進去
CREATE INDEX idx_posts_user_include
ON posts(user_id) INCLUDE (title, created_at);
```

涵蓋索引能讓查詢快得多，因為 PostgreSQL 不需要碰表格，直接從索引就能滿足整個查詢。代價是索引佔用更多空間，寫入也會稍微慢一點。

部分索引（Partial Index）

buildmoat.org

有時候你只需要索引資料的一個子集。例如在我們的社群媒體平台，大多數查詢可能只關心活躍用戶，而不是已刪除的帳號：

```
-- 一般索引對所有資料都建立索引
CREATE INDEX idx_users_email ON users(email);  -- 索引所有用戶

-- 部分索引只對活躍用戶建立索引
CREATE INDEX idx_active_users
ON users(email) WHERE status = 'active';  -- 更小、更快
```

部分索引在以下情況特別有效：你的大多數查詢只需要資料的某個子集、你有大量「非活躍」或「已刪除」的記錄不需要索引、或者你想降低索引的整體大小和維護開銷。

實際的效能基準

在面試的非功能性需求討論中，你很可能會設定一些延遲目標。以下是粗略的參考數字（實際數字會因硬體和工作負載而有很大差異，但足以讓你在面試中做出合理的估算）：

查詢效能：

- 簡單的索引查找：每個核心每秒數萬次
- 複雜的 JOIN 查詢：每個核心每秒數千次
- 全表掃描：高度取決於資料是否能放進記憶體

規模限制：

- 超過 1 億筆資料後，表格開始變得難以管理
- 全文搜尋在幾千萬份文件以內運作良好
- 超過 1 千萬筆資料的表格做複雜 JOIN 開始吃力
- 工作集（working set）超過可用 RAM 後效能急劇下降

這些不是硬性上限——有了適當的優化，PostgreSQL 可以處理更多。但這些是你應該開始考慮分區（partitioning）、分片（sharding）或其他擴展策略的時機點。

記憶體是效能的關鍵。 能從記憶體滿足的查詢，比需要存取磁碟的查詢快好幾個數量級。一個基本原則：盡量讓你的工作集（頻繁存取的資料）保持在 RAM 裡。

寫入效能

讀取可能佔大多數工作負載，但寫入效能往往更關鍵，因為它直接影響用戶體驗——沒有人想在按下「發布」之後等好幾秒才看到內容出現。

PostgreSQL 寫入的運作方式

當一筆寫入發生時，PostgreSQL 會經歷以下幾個步驟來確保效能和持久性：

1. **緩衝快取 + WAL 記錄（記憶體中）**：寫入發生時，PostgreSQL 修改記憶體中的緩衝快取（把它標記為「髒頁」），同時產生一筆 WAL（Write-Ahead Log，預寫日誌）記錄。這兩件事都在記憶體中發生。

buildnoat.org

2. **WAL 刷寫到磁碟（交易提交時）**：提交時，WAL 記錄被寫進磁碟。這是循序寫入，相對較快。WAL 落盤後，這筆交易就算是持久化了——即使伺服器崩潰，PostgreSQL 也能透過重播 WAL 來恢復已提交的變更。
3. **背景寫入程序（記憶體 → 磁碟，非同步）**：記憶體裡的髒頁會定期被寫入實際的資料檔案。這是非同步進行的，由背景寫入程序處理，讓 PostgreSQL 能把多個變更批次在一起提升效能。
4. **索引更新（記憶體和磁碟）**：每個索引都需要更新以反映變更。這也是為什麼有太多索引會大幅降低寫入速度——每個索引都需要額外的 WAL 記錄和記憶體更新。

這個架構讓 PostgreSQL 的寫入可以很快——大部分工作在記憶體中發生，同時透過 WAL 確保持久性。

吞吐量限制

在一台硬體還不錯（但不是頂尖）的機器上，調校良好的 PostgreSQL 實例大約可以處理：

- 簡單的 INSERT：每個核心每秒約 5,000 筆
- 有索引更新的 UPDATE：每個核心每秒約 1,000 到 2,000 筆
- 複雜交易（涉及多張表和多個索引）：每秒幾百筆
- 批次操作：每秒幾萬筆

影響這些限制的因素：

- **硬體**：寫入吞吐量通常被 WAL 的磁碟 I/O 卡住
- **索引數量**：每多一個索引，寫入吞吐量就減少一些
- **複製**：如果設定了同步複製，要等 replica 確認才能回應，延遲會增加
- **交易複雜度**：觸碰的表格和索引越多，速度越慢

注意：這些是單節點的數字。 如果你的系統需要超過每秒 5,000 筆的寫入，這不代表 PostgreSQL 不行，只是代表你需要把資料分片到多個節點上。

寫入效能優化策略

假設單節點大約能處理每秒 5,000 筆寫入，如果需要更高的吞吐量，有哪些選項？

1. 垂直擴展（Vertical Scaling）

在跳到複雜的解法之前，可以先考慮升級硬體：更快的 NVMe 磁碟改善 WAL 效能、更多 RAM 增加緩衝快取大小、更多核心的 CPU 提升並行處理能力。這通常不是面試裡最精彩的答案，但是個合理的起點。

2. 批次處理（Batch Processing）

最簡單的優化是把多個寫入放在一起處理。與其逐一執行每個寫入，不如把多個操作集中在一個交易裡執行。舉例來說，與其一次一次插入 1,000 個按讚，不如一次插完：

```
-- 取代 1000 次獨立的 INSERT：
INSERT INTO likes (post_id, user_id) VALUES
  (1, 101), (1, 102), ..., (1, 1000);
```

這表示你在把寫入先暫存在伺服器的記憶體裡，再批次提交到磁碟。明顯的風險是：如果在批次中途崩潰，那一批的所有寫入都會遺失。

buildmoat.org

3. 寫入卸載（Write Offloading）

有些寫入不需要同步發生。分析資料、活動日誌、聚合指標，通常都可以非同步處理。與其直接寫入 PostgreSQL，可以：

- 把寫入傳到訊息佇列（例如 Kafka）
- 讓背景 worker 批次處理這些佇列中的寫入
- 視需要維護一個獨立的分析資料庫

這個模式特別適合活動日誌記錄、分析事件、指標聚合，以及不需要即時更新的非關鍵資料（例如「最後上線時間」）。這類寫入不需要馬上發生，可以在背景處理，不影響核心用戶體驗。

4. 表格分區（Table Partitioning）

對於大型表格，分區可以把資料拆分到多個實體表格，同時改善讀取和寫入效能。最常見的是依時間分區：

```
CREATE TABLE posts (
    id SERIAL,
    user_id INT,
    content TEXT,
    created_at TIMESTAMP
) PARTITION BY RANGE (created_at);

-- 按月建立分區
CREATE TABLE posts_2024_01 PARTITION OF posts
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');
```

為什麼這對寫入有幫助？首先，不同的資料庫連線可以同時寫入不同的分區，提升並行度。其次，資料插入時索引更新只需要發生在相關的分區，而不是整個表格。對讀取也有幫助——用戶查看最近的貼文時，PostgreSQL 只需要掃描最近的分區，不需要翻遍幾年的歷史資料。

一個常見的做法是把最近的分區放在快速儲存（例如 NVMe 磁碟），把較舊的分區移到便宜的儲存。用戶對最近的資料存取最快，而這也正是他們最在乎的。

5. 分片（Sharding）

當單節點不夠用時，分片讓你把寫入分散到多個 PostgreSQL 實例上。這在面試中是最常被提到的解法。你需要說清楚要按什麼欄位分片、以及如何分配資料。

例如，我們可以按 `user_id` 對貼文做分片，讓一個用戶的所有資料都住在同一個分片上。這很重要——讀取資料時，我們想要避免跨分片查詢（scatter-gather），需要同時從多個分片拉資料。

分片的原則是選擇你查詢最頻繁的那個欄位。如果你最常查詢的是「某個用戶的所有貼文」，就按 `user_id` 分片。

分片帶來了複雜度——你需要處理跨分片查詢、維護跨分片一致的 schema、以及管理多個資料庫。只有在較簡單的優化不夠用時才引入它。

和 DynamoDB 不同，PostgreSQL 沒有內建的分片解法，需要自己實作，這有一定的挑戰。另一個選擇是使用 Citus 這樣的托管服務，它幫你處理了很多分片的複雜性。

複製（Replication）

bubdinoat.org

我們已經討論了如何在單節點上優化寫入效能，但現實世界的部署幾乎都會用到複製，主要有兩個原因：

- **擴展讀取**：把查詢分散到多個 replica 上
- **高可用性**：當節點故障時提供備援

同步 vs 非同步複製

同步複製：primary 節點要等 replica 確認收到資料後，才把寫入成功回傳給客戶端。提供更強的一致性，但延遲更高。

非同步複製：primary 節點立刻把寫入成功回傳給客戶端，在背景把變更複製到 replica。提供更好的效能，但 replica 可能短暫落後於 primary。

很多組織採用混合方式：保留少數幾個同步 replica 確保強一致性，同時維護額外的非同步 replica 做讀取擴展。PostgreSQL 讓你指定哪些 replica 是同步的。

擴展讀取

複製最常見的用途就是擴展讀取效能。透過建立讀取 replica，你可以把讀取查詢分散到多個資料庫實例，所有寫入則走 primary。這特別有效，因為大多數應用程式是讀多寫少的。

回到我們的社群媒體例子：用戶瀏覽動態消息或查看個人頁面，都是可以由任何 replica 處理的讀取操作。只有建立貼文或更新個人資料時才需要用到 primary。這樣讀取吞吐量就乘以了 replica 的數量 N。

讀取 replica 有一個要注意的地方：**複製延遲（replication lag）**。如果用戶做了一個變更後立刻讀取，他可能看不到自己的變更——因為他打到的 replica 還沒跟上最新狀態。這叫做「讀自己寫（read-your-writes）」一致性問題。解決方法通常是讓剛寫入的請求短暫地路由到 primary，或者在應用層處理這個不一致。

高可用性

複製的第二個主要好處是高可用性。透過在多個節點上保存資料副本，你可以在硬體故障時不中斷服務。如果 primary 節點掛掉，其中一個 replica 可以被提升為新的 primary。

這個故障切換（failover）過程通常包含：偵測 primary 已下線、把一個 replica 提升為 primary、更新連線資訊、讓應用程式指向新的 primary。

大多數團隊使用托管的 PostgreSQL 服務（例如 AWS RDS 或 GCP Cloud SQL），這些服務自動處理了故障切換的複雜性。在面試中，知道故障切換是可行的、以及大致上它怎麼運作，就已經足夠了。

資料一致性

如果你在非功能性需求中選擇了優先考慮一致性而非可用性，PostgreSQL 是個強力的選擇。它從設計之初就提供強一致性保證，透過 ACID 交易實現。但光是選了 PostgreSQL 還不夠——你需要理解如何真正利用這些 ACID 屬性來解決你的一致性需求。

面試中常見的錯誤是說「我們用 PostgreSQL 因為它支援 ACID」，卻說不清楚你具體怎麼利用這些 ACID 屬性來解決問題。

交易（Transactions）

交易是面試中最常被討論到的主題之一。交易是一組必須一起成功或一起失敗的操作。這是確保 PostgreSQL 一致性的基礎。

buildmeat.org

舉一個簡單的例子：在兩個銀行帳戶之間轉帳。我們需要確保從一個帳戶扣款的同時，另一個帳戶也完成入帳，兩個操作不能只做一半：

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
```

這個交易確保了原子性——兩個更新皆發生，或者皆不發生。

並行操作的一致性

交易確保了單一操作序列的一致性，但當多個交易同時發生時，事情就複雜了。在大多數真實應用裡，你會有多個用戶或服務同時試圖讀取和修改資料。

這是很多面試者在面試中卡關的地方。他們理解基本的交易，但沒有想清楚多個操作同時發生時如何維持一致性。

來看一個拍賣系統的例子。用戶對商品出價，我們只接受高於當前最高出價的新出價。單一交易能確保「查看當前最高價」和「出新的價」這兩步驟是原子性的，但如果兩個用戶同時出價呢？

```
BEGIN;
-- 取得商品 123 的當前最高出價
SELECT maxBid from Auction where id = 123;

-- 如果新出價更高，則插入
INSERT INTO bids (item_id, user_id, amount)
VALUES (123, 456, 100);

-- 更新最高出價
UPDATE Auction SET maxBid = 100 WHERE id = 123;
COMMIT;
```

buildmoat.org

就算包在交易裡，在 PostgreSQL 的預設隔離層級（Read Committed）下，如果兩個用戶同時出價，還是可能出現一致性問題——兩個交易都可能在另一個提交之前讀到相同的最高出價。

這會怎麼導致不一致的狀態？

1. 用戶 A 的交易讀到當前最高出價：90 元
2. 用戶 B 的交易讀到當前最高出價：90 元
3. 用戶 A 出價 100 元，提交
4. 用戶 B 出價 95 元，提交
5. 現在我們有了一個不合法的狀態：95 元的出價被接受了，即使之前已經有 100 元的出價！

有兩個主要的解法：

解法一：列層級加鎖（Row-Level Locking）

最簡單的解法是在檢查和更新出價時鎖定那一行。透過 `FOR UPDATE` 子句，我們告訴 PostgreSQL 鎖定我們正在讀取的行。其他試圖用 `FOR UPDATE` 讀取這些行的交易，必須等到我們的交易完成才能繼續：

```
BEGIN;
-- 鎖定這筆拍賣記錄，並取得當前最高出價
SELECT maxBid FROM Auction WHERE id = 123 FOR UPDATE;

-- 如果新出價更高，則插入
INSERT INTO bids (item_id, user_id, amount)
VALUES (123, 456, 100);

-- 更新最高出價
UPDATE Auction SET maxBid = 100 WHERE id = 123;
COMMIT;
```

在面試中，不要只說「我們用交易」。更準確的說法是：「我們用交易加上對拍賣記錄的列層級加鎖」。

解法二：更高的隔離層級

另一個選擇是使用更嚴格的隔離層級：

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
-- 相同的程式碼...
COMMIT;
```

可序列化隔離（Serializable）能防止所有一致性異常，但代價是：如果兩個交易衝突，其中一個會被回滾，應用程式需要有重試邏輯。

buildmoat.org

PostgreSQL 的隔離層級

PostgreSQL 支援三個實際有效的隔離層級：

Read Committed（預設）：只看到在查詢開始之前已提交的資料。同一個交易裡，每次查詢都能看到其他交易在它開始後才提交的新資料。效能好，但可能發生「不可重複讀」（同一個查詢在交易內返回不同的結果）。

Repeatable Read：PostgreSQL 的實作比 SQL 標準的要求更強。它建立了一個在交易開始時的一致性快照，同時防止「不可重複讀」**和**「幻讀（phantom reads）」——即使其他交易提交了符合你查詢條件的新行，你也看不到它們。PostgreSQL 在這個層級的保證比其他資料庫更強，很多你在別的資料庫需要 Serializable 才能處理的情況，在 PostgreSQL 用 Repeatable Read 就夠了。

Serializable：最強的隔離層級，讓交易表現得像是依序執行一樣。防止所有類型的並行異常，但需要應用程式有重試邏輯來處理交易衝突。

	隔離層級	髒讀	不可重複讀	幻讀	序列化異常
Read Uncommitted	理論上允許，但 PG 實際等同 Read Committed	不可能	可能	可能	可能
Read Committed	—	不可能	可能	可能	可能

	隔離層級	髒讀	不可重複讀	幻讀	序列化異常
Repeatable Read	—	不可能	不可能	不可能（PG 特有）	可能
Serializable	—	不可能	不可能	不可能	不可能

什麼時候用列層級加鎖、什麼時候用更高的隔離層級？

	可序列化隔離	列層級加鎖
並行度	較低，衝突時需要重試	較高，只有碰到同一行才衝突
效能開銷	較高，需要追蹤所有讀寫依賴	較低，只追蹤特定的鎖
適用情境	複雜交易，難以預先知道要鎖哪些行	清楚知道哪些行需要原子更新
錯誤處理	需要處理序列化失敗（重試）	需要處理死鎖情境
典型例子	跨多張表的複雜財務計算	拍賣出價、庫存更新

一般來說，當你確切知道需要鎖定哪些行時，優先選列層級加鎖。把可序列化隔離留給那些複雜到難以推理該鎖什麼的情況。

什麼時候選 PostgreSQL（以及什麼時候不選）

PostgreSQL：

- 提供強大的 ACID 保證，同時透過複製和分區有效擴展
- 透過 JSONB 支援同時處理結構化和非結構化資料
- 內建全文搜尋和地理空間查詢，消除了對額外系統的需求
- 透過複製有效擴展讀取能力
- 有出色的工具生態系統和成熟的社群

從 PostgreSQL 出發，然後說明你為什麼需要偏離它。這比從一個小眾解決方案出發、再試圖證明它比 PostgreSQL 更好，要有說服力得多。

PostgreSQL 的強項

PostgreSQL 在以下場景特別出色：

- 資料之間有複雜的關係
- 需要強一致性保證
- 需要豐富的查詢能力
- 資料混合了結構化和非結構化部分（JSONB）
- 需要內建的全文搜尋
- 需要地理空間查詢

完美的應用場景包括：電商平台（庫存、訂單、用戶資料）、金融系統（交易、帳戶、稽核日誌）、內容管理系統（貼文、留言、用戶）、分析平台（在合理規模內）。

什麼時候考慮替代方案

1. 極端的寫入吞吐量

buildmoat.org

如果你需要處理每秒數百萬次的寫入，PostgreSQL 會遇到瓶頸——每一筆寫入都需要一筆 WAL 記錄和索引更新，即使是最快的儲存也會有 I/O 瓶頸。即使加上分片，協調多個 PostgreSQL 節點的寫入也會帶來複雜度和延遲。這種情況可以考慮：

- NoSQL 資料庫（如 Cassandra）用於事件串流
- 鍵值儲存（如 Redis）用於即時計數器

2. 全球多區域需求

當你需要在多個區域同時接受寫入的主動-主動（active-active）部署時，PostgreSQL 有根本的限制。它的單一主節點架構意味著只能有一個 Region 是主要的寫入節點，其他只能是讀取 replica。真正的跨 Region 同時寫入會帶來嚴重的資料一致性和衝突解決問題，PostgreSQL 根本不是為這種情境設計的。這種情況可以考慮：

- CockroachDB：提供全球 ACID 合規
- Cassandra：提供全球規模下的最終一致性
- DynamoDB：提供托管的全球資料表

3. 簡單的鍵值存取模式

如果你的存取模式就是純粹的鍵值（只是按鍵存取和取值，不需要 JOIN 或複雜查詢），PostgreSQL 是殺雞用牛刀。它的 MVCC 架構、WAL 日誌、複雜的查詢規劃器帶來了你根本不需要的額外開銷。這種情況可以考慮：

- Redis：記憶體中的超高效能
- DynamoDB：托管的高擴展性
- Cassandra：寫入密集型工作負載

> **重要提醒**：單純「需要擴展」本身，**不是**棄用 PostgreSQL 的好理由。設計得當的 PostgreSQL 能處理相當大的規模。

buildmoat.org

總結

在面試中討論 PostgreSQL 時，聚焦在分析具體的需求——資料一致性、查詢模式、規模——而不是跟著流行走。準備好討論關鍵的取捨：ACID vs 最終一致性、讀取 vs 寫入的擴展策略、以及索引設計決策。從簡單的方案出發，只在有需要時才增加複雜度。

PostgreSQL 豐富的功能集通常能消除對額外系統的需求：它的全文搜尋能力可能取代 Elasticsearch；JSONB 支援可能省掉 MongoDB；PostGIS 能處理原本需要專門地理空間資料庫的需求；內建複製通常能提供足夠的擴展能力。但同樣重要的是認清 PostgreSQL 不是最佳選擇的情況——例如需要極端寫入擴展或全球分散式部署時，Cassandra 或 CockroachDB 可能更合適。

附錄：基礎 SQL 概念

在深入 PostgreSQL 特有的功能之前，讓我們複習一下 SQL 資料庫如何組織資料。以下這些核心概念適用於任何 SQL 資料庫，是理解整篇講義的基礎。

關聯式資料庫的基本原理

PostgreSQL 的核心是把資料存在表格（table）裡，也叫做關係（relation）。把表格想成一個試算表，有欄位（column）和列（row）。每個欄位有特定的資料型別（例如文字、數字、日期），每一列代表

一筆完整的記錄。

舉個具體的例子。假設我們在設計一個社群媒體平台，用戶表格（`users`）可能長這樣：

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

新用戶註冊時，我們在這個表格裡建立一筆新列。每個用戶有唯一的 `id`（這就是 `PRIMARY KEY` 的意思），而且我們確保沒有兩個用戶能有相同的 `username` 或 `email`（這就是 `UNIQUE` 的意思）。

但用戶需要能夠發布內容。這就是「關聯式」的意義。我們建立一個連接到用戶的 `posts` 表格：

```
CREATE TABLE posts (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),
    content TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

看到 `REFERENCES users(id)` 了嗎？這叫做**外鍵（foreign key）**——它在 `posts` 和 `users` 之間建立了一個關係。每篇貼文都必須屬於一個合法的用戶，PostgreSQL 會幫我們強制執行這個規則。這是關聯式資料庫的一個核心優勢：透過強制執行這些關係來維護資料完整性。

在面試中，能夠說明這些關係很關鍵。主要有三種類型：

- **一對一（One-to-One）**：例如一個用戶和他們的個人設定
- **一對多（One-to-Many）**：例如一個用戶可以有很多篇貼文
- **多對多（Many-to-Many）**：例如用戶和他們按讚的貼文（一個用戶可以對多篇貼文按讚，一篇貼文也可以被多個用戶按讚）

多對多關係用一個**關聯表（join table）**來處理：

```
CREATE TABLE likes (
    user_id INTEGER REFERENCES users(id),
    post_id INTEGER REFERENCES posts(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id, post_id)
);
```

這種把資料拆分到不同表格、透過關係連接的結構，叫做**正規化（normalization）**。它讓我們避免重複資料（不需要在每篇貼文裡存用戶資訊）、維護資料完整性（用戶改了名字，所有地方都跟著更新）、讓資料模型更靈活（新增用戶屬性不需要動到貼文表格）。

雖然正規化通常是好的，但有時候我們會刻意做反正規化（denormalization）來換取效能。例如，我們可能把貼文的按讚數直接存在 `posts` 表格裡，即使我們可以從 `likes` 表格計算出來。這種資料一致性和

buildneat.org

查詢效能之間的取捨，正是面試中你應該主動討論的內容。

ACID 特性

PostgreSQL 最大的優勢之一是它嚴格遵守 ACID（原子性、一致性、隔離性、持久性）特性。如果你用過 MongoDB 或 Cassandra，你熟悉最終一致性或寬鬆的交易保證，這是 NoSQL 資料庫常見的取捨。PostgreSQL 採取不同的做法——它確保資料始終遵循所有定義的規則和約束，所有交易完整完成或完全不發生，即使這意味著犧牲一些效能。

讓我們用一個轉帳的真實例子來說明 ACID 的每個特性：

原子性（Atomicity）——全部成功或全部失敗

從儲蓄帳戶轉 100 元到支票帳戶涉及兩個操作：從儲蓄帳戶扣除、加入支票帳戶。原子性保證要嘛兩個操作都成功，要嘛都不發生。如果系統在扣除之後、加入之前崩潰，PostgreSQL 會回滾整個交易。錢不會憑空消失。

```
BEGIN;
  UPDATE accounts SET balance = balance - 100 WHERE account_id = 'savin
gs';
  UPDATE accounts SET balance = balance + 100 WHERE account_id = 'check
ing';
COMMIT;
```

一致性（Consistency）——資料完整性

一致性確保交易只能把資料庫從一個合法的狀態帶到另一個合法的狀態。例如，如果我們設定了帳戶餘額不能為負的規則，PostgreSQL 會拒絕任何會讓餘額變成負數的交易：

```
CREATE TABLE accounts (
    account_id TEXT PRIMARY KEY,
    balance DECIMAL CHECK (balance >= 0),
    owner_id INTEGER REFERENCES users(id)
);
```

注意：ACID 裡的「一致性」和 CAP 定理裡的「一致性」意義略有不同。ACID 裡的一致性指的是資料始終遵循所有定義的規則和約束；CAP 定理裡的一致性指的是所有節點同時看到相同的資料。

隔離性（Isolation）——並行交易

隔離層級決定了交易如何與其他並行交易正在修改的資料互動，這在本篇講義稍早已詳細討論過。

持久性（Durability）——永久儲存

一旦 PostgreSQL 說交易已提交，這些變更就保證是持久的，即使發生崩潰或停電也不會丟失。這是透過 WAL（Write-Ahead Logging）實現的：變更先寫入日誌，日誌刷寫到磁碟，然後才算交易完成。

雖然持久性是有保證的，但也有效能成本。有些應用程式可能選擇為了速度而放寬持久性（例如設定 `synchronous_commit = off`），這意味著某些尚未寫入磁碟的寫入在停電時可能丟失。

ACID 為什麼重要

在面試中，你常常需要在不同類型的資料庫之間做選擇。幾個思考方向：

- **金融交易**：絕對需要 ACID 特性，防止錢被弄丟或重複扣款

buildmoat.org

- **社群媒體按讚**：可以接受最終一致性
- **用戶認證**：可能需要 ACID 防止安全問題
- **分析資料**：可能優先考慮效能而非嚴格一致性

SQL 語言基礎

雖然在系統設計面試中很少需要真正寫 SQL 查詢，但理解 SQL 的能力有助於你做出更好的架構決策。SQL 指令大致分為四類：

DDL（資料定義語言）——建立和修改資料庫結構：

```
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  email VARCHAR(255) UNIQUE
);

ALTER TABLE users ADD COLUMN username TEXT;
```

DML（資料操作語言）——管理表格中的資料：

```
-- 找出過去一週加入的所有用戶
SELECT * FROM users
WHERE created_at > NOW() - INTERVAL '7 days';

-- 更新用戶的 email
UPDATE users SET email = 'new@email.com'
WHERE id = 123;
```

DCL（資料控制語言）——控制存取權限：

```
-- 給特定用戶只讀權限
GRANT SELECT ON users TO read_only_user;
```

TCL（交易控制語言）——管理交易：

```
BEGIN;
  -- 多個操作...
COMMIT;
```

在面試中，你可能被問到的是資料的存取模式，而不是具體的 SQL 語法。例如：「你會怎麼高效率地查詢這些資料？」或「你會建立哪些索引？」這些問題考驗的是你對資料庫概念的理解，而不是對 SQL 語法的熟記。

buildmoat.org